Centher dashboard rollout plan for the Grounding System

Plan only. This document does not write code, merge, deploy, or touch production data or accounts. Implementation starts in the Centher chat from this plan, not in this document.

Goal

Bring the Grounding System into the Centher dashboard app, using the shared contract from 00-the-grounding-system.md, without changing Centher's stack. The concrete outcomes are: a dependency graph and blast-radius script for the Centher frontend; a reference checker for any AI-generated text in the app; spec discipline rules for the Centher crew; and a tenant scoping audit for org, team, and admin boundaries. The plan also uses the graph to support the explainer program: it orders explainer material shared-components-first and gives new agents an onboarding path that does not require reading all 75 screens.

Verified current state

Everything in this section was checked from files on disk during the writing of this plan. Anything not checked is listed under Assumptions, not stated as fact.

Repo and stack

- Repo: ~/workspace/centher, GitHub repo irfansaeedkhan/centher.dashboard.fullstack.
- Current main at check time: 5c8a331c, merge of PR #32 (chat completeness). The working tree state beyond that commit was not verified in this plan.
- Framework: Next.js 13.3.4 with the pages router (routes are pages/*.page.tsx, API routes are pages/api/[[...route]].api.ts). React 18.2. TypeScript 5.0.4. Path alias @/ maps to the repo root (tsconfig paths).
- Auth: Better Auth 1.7.7, in lib/auth/better-auth.ts. No Clerk, no Supabase auth. This plan changes neither.
- Database: Drizzle ORM 0.45.4 with the Neon serverless driver, in db/index.ts and db/schema.ts. Drizzle migrations 0001 through 0010 cover NFT trading, media, chat, typing, team bots, grill fixes, and chat completeness.
- API: Hono app in server/hono/app.ts, served through the pages API catch-all. The explainer inventory counts 247 route registrations in that file.
- Tests: Jest, with test files under **tests**/ (chat completeness, auth trap hardening, feed grill fixes, social routes, and others).
- Package manager: yarn.lock is present. Scripts include dev, build, start, lint, test, db:push, db:seed (tsx scripts/seed.ts), and bots:respond (tsx scripts/bot-responder.ts).

- Size: 827 TypeScript and TSX files under components, pages, lib, server, store, hooks, utils, models, and constants. 237 files under components. 391 files under pages.
- ts-morph is not in package.json. The G1 script will add it as a dev-only tool, not as an app runtime dependency. That is a tooling addition, not a stack change. If the Centher chat objects, the same contract can be built with the TypeScript compiler API directly, and that decision belongs in the Phase 1 spec.

What was checked about model calls

- A search for model provider terms (openai, anthropic, gemini, groq, openrouter, generativelanguage, huggingface, replicate, fal.ai) across lib, server, components, pages, and scripts returned only false positives (variable names such as allMems and allMessages). No real model call site was found in the app code.
- The team bot responder (lib/bots/responder.ts and scripts/bot-responder.ts) is explicitly deterministic: its header states no network calls, no secrets, no side effects. Given a bot role and a human message, it returns 1 to 3 sentences by hash-picked template variant. It is not a model call.
- The voice token module (lib/voice/voice-token.ts) mints short-lived JWTs for the Centher voice server (VoiSpace rooms), with a 15 minute TTL and a fail-closed rule when the secret is missing. It is an auth token, not a model call.
- Conclusion for this plan: Centher has no real model call sites today. G3 is therefore deferred, with a trigger condition defined below. This is a verified finding from code search, not an assumption.

Explainer program state

- Inventory: centher-explainer-inventory.md maps 75 routes (37 WORKING, 21 PARTIAL, 10 PLACEHOLDER, 7 REDIRECT) and 65 modals, 140 items total. The inventory was built from code at main commit 8a33d5f1, which is behind current main 5c8a331c. The inventory itself states it used code evidence only, with no live browser check, so no item in it is claimed as live verified.
- Script: centher-explainer-script.md, 19 segments.
- Coverage checklist: centher-explainer-coverage-checklist.md records that footage for all 19 segments exists, and that the final video was delivered on 2026-10-29 morning as centher-explainer-final.mp4 (H.264, 1280x720, 775.03 seconds, burned-in captions, 105 captions), plus a soft-subtitle variant and sidecar SRT, in the goal files folder.
- Honest qualifiers recorded in the checklist (not clean passes): post edit, archive, and delete controls were not reachable in the takes; comment edit and delete controls were not found on hover; marketplace listing controls are disabled with on-screen notices; staking detail pages (pool details, project details, referrals) were not opened; the cookies dialog did not appear on a fresh visit; several modals are caption-described or attempted rather than individually verified. Scratch posts, comments, and chat messages from recording were deleted after recording through the app delete API. Deleted content renders as tombstone placeholders by design.

- Gating facts from the inventory that constrain any live work: signed out visitors to authenticated pages go to /auth/login; admin pages require an admin check (middleware list plus AdminGuard calling GET /api/admin/check); citizen-only pages are /marketplace/create, /marketplace/create-collection, /settings/citizen/team-members, and /staking/create-staking; /network-rewards/overview and /network-rewards/rewards redirect to /coming-soon in production; /stream and /ui-test rewrite to /404 in production; /launchpad redirects to a fixed round URL which itself redirects to the launchpad list.
- Live site: https://centher-app.vercel.app/ (stated in the inventory).

Demo accounts

- Verified: scripts/seed.ts line 22 reads NEXT_PUBLIC_DEMO_EMAIL or defaults to demo@centher.io. The inventory confirms the login form shows a demo line and prefilled demo values, and that the values must match scripts/seed.ts. Sign-in posts to /api/auth/sign-in/email and goes to /feed.
- Not verified in the files checked for this plan: a second demo account. See Assumptions. Phase 0 must confirm it before any two-account flow is planned.

Standing doctrine already in the Centher goal

- The Centher LEARNING.md already carries the standing doctrine: real-browser record plus analyze rule, speed gate, UI/UX alignment gate, UI/UX screen plus modal review rule, expert video and audio analysis rule, and the post-recording re-analyze and re-record loop. This plan inherits all of them. It does not restate them as new rules.
- The Centher LEARNING.md also records real failure cases the G1 script must respect: fictional UI state with no persisted row behind it; the neon-http driver having no transaction support (atomicity means single-statement data-modifying CTE chains); enum vocabulary drift between API mappers and UI branches; and committed secrets living in more than one file.
- Discrepancy noted: the Centher GOAL.md still describes a 5-agent audit loop. This plan follows the standing 12-agent crew decision. The Centher chat should update GOAL.md to the 12-agent crew when implementation starts, so the goal doc and the plan stop disagreeing.

Assumptions (unverified, not facts)

1. A second demo account exists for two-account flows (for example, a second account for chat presence and group tests). It was not found in the goal files checked for this plan. Phase 0 confirms it or creates a throwaway second account under the same demo rules. No phase that needs two accounts starts until this is confirmed.
2. VoiSpace live audio and video working state is unverified. The components exist under components/voispace/, and the explainer treated VoiSpace as note-only (channel rows and messages only, no live video feed shown). The standing rule applies: hide the VoiSpace button if audio or video is not working, keep it if it works. No phase in this plan assumes VoiSpace works live.

3. The DayMirror core implementation had not landed as a file when this plan was written. Only 00-the-grounding-system.md was present in ~/workspace/your_files/grounding-system/. The DayMirror plan file listed in that doc was not on disk. This plan therefore reuses the shared contract from 00, not a DayMirror code file. Phase 0 confirms the DayMirror core status. If the DayMirror core lands first, Centher ports it. If Centher Phase 1 lands first, it is built to the shared contract, and the DayMirror chat adopts the same shape. No component is built twice with a different contract.
4. The DayMirror core is assumed to follow the same contract in 00 (graph JSON shape, four G2 gates, gateway contract). If the DayMirror implementation diverges from 00, the divergence is resolved in the Centher chat before Phase 1 code is written, and 00 is corrected.
5. Demo data seeded by scripts/seed.ts is assumed to still match the login box values. Phase 0 verifies by signing in, not by reading the seed file alone.

Scope in

- G1 dependency graph and blast-radius script for the Centher frontend, built to the shared contract in 00.
- A screen-to-component matrix for the 75 routes, a shared-components-first ordering map for explainer material, and an onboarding guide for new agents.
- G2 reference checker library with Centher allowed sets, plus tests, plus wiring at the one place generated text reaches users today (the bot reply path) as a proven guard.
- G4 spec discipline rules adopted into Centher crew docs, proven on one real slice.
- G5 tenant scoping audit for org, team, and admin boundaries, with a two-tenant test on demo or staging accounts.
- LEARNING.md updates per phase, a learning note and PDF per definition of done, and test-data cleanup with ids recorded.
- UI/UX review of every screen and modal touched by this plan, through the UI/UX skills with local-ui-ux-director sign-off, before it is cleared.

Scope out

- No stack change. No LangSmith, no Clerk, no Supabase, no new vendor, unless an app already uses it. Centher keeps Better Auth, Drizzle, and Neon.
- No visual graph canvas in version 1. The script comes first. A folding navigation view is a later phase, only after the script proves its value.
- No semantic caching. G3 is deferred entirely (see below), and semantic caching is deferred everywhere in version 1 by the shared system.
- No quality judgments about code. The graph answers structure questions (what imports what, what breaks), not quality questions (is this code good).
- No on-chain transaction completed on camera or in a Pilot. On-chain steps are shown as form and dialog only, with the on-camera note that the step is not completed. This matches the explainer rule.

- No production data touched. Demo, staging, and throwaway accounts only, until a phase explicitly says otherwise with its own approval.
- No rebuild of the DayMirror core from scratch. Centher implements the shared contract. If the DayMirror implementation exists by Phase 1, Centher ports it and adapts only the repo-specific parts (pages router file naming, path alias, component roots).
- No replacement of the explainer program. This plan supports it. See the relationship section.

Components adopted

G1: Dependency graph and blast radius (adopted, Phase 1 and 2)

Reuse, not rebuild: the graph JSON shape, edge kinds, coverage reporting, query names (blast-radius, dependency-chain, affected-tests), and regression rule come from 00. The Centher implementation adapts only what is repo-specific:

- File roots: components/, pages/ (including pages/_components/), lib/, server/, store/, hooks/, utils/, models/, constants/. Web3 and onchain code under web3/ is included in the graph because it is TypeScript, but onchain behavior is never claimed from the graph alone.
- File naming: pages router suffixes (.page.tsx for routes, .api.ts for API routes) must be normalized so a route file and its imports resolve to the same node. The Phase 1 spec names the normalization rule explicitly.
- Path alias: @/* maps to the repo root. The parser must resolve the alias through tsconfig paths, not by guessing.
- Edge kinds: plain imports, re-exports, dynamic imports with a literal string, require with a literal path. Non-literal require calls are reported unresolved with a reason.
- Coverage is reported next to every result, always: files found, files parsed, files skipped with reasons, imports resolved, imports failed with reasons.
- Queries: blast-radius(file, depth 2), dependency-chain(file, depth 2), affected-tests(changed files). Depth 2 by default, because further levels return half the repo.

Uses in Centher, in order of value:

1. Blast radius before shared-component changes. Centher has 237 component files serving 75 routes. A change to a shared post card, chat composer, modal shell, or profile header touches many screens. The graph lists them before the change, instead of an agent discovering them by reading code.
2. Explainer ordering. The graph ranks components by fan-in (how many screens import them). Explainer material is then ordered shared-components-first: the shared pieces are explained once, and each screen segment references them, instead of re-explaining the same modal shell in every segment. This is a support input to the explainer program, not a new video.
3. Onboarding. A new agent reads the top shared components and their screens from the graph, plus the screen-to-component matrix, instead of reading all 75 screens. The onboarding guide names the components to read and the screens each one serves.

4. Affected-area testing. Given changed files, the reverse walk finds every test that transitively imports them. That is how affected-area testing stops being guesswork, and it plugs directly into the existing Jest suite.

Later phase, not the first step: folder folding for navigation by feature rather than by file. Small folders merge upward until the map is manageable, and clicking a folded folder lists its files. This is Phase 6, only after the script is proven and only if the Centher chat agrees it is worth a UI surface.

G2: Reference verification (adopted, Phase 3)

The checker is deterministic. It needs no model and costs no tokens. It runs over generated text before that text reaches a user or a Judge. On failure, the output is flagged, not shipped. The checker returns the failed gate, the offending citation, and the allowed set it was checked against. It never rewrites output silently. Repair is a separate, visible step.

Centher allowed sets, per gate:

- Gate 1, evidence existence: component names come from the G1 graph (the exact file list the parser found). Screen names come from the 75 routes in the inventory, rebased to current main in Phase 0. Record ids (post ids, comment ids, conversation ids, message ids, NFT ids, collection ids, pool ids, user ids) come from a database read performed for that request, never from the text being checked.
- Gate 2, entity allowlisting: entity names come from the app's own lists for that request (seeded team member names, collection names, pack names). Paraphrased names from training data fail.
- Gate 3, number binding: numbers (counts, prices, APY values, days, balances) must exactly match a canonical value surfaced from the app's own data for that request. Rounded, sign-flipped, or reformatted numbers fail. Marketplace prices in any spoken or shown material carry the demo trading number only rule from the explainer checklist.
- Gate 4, claim-type rules: a comparison claim must cite the two things compared. A recommendation must cite the evidence it rests on. A claim about a screen must cite that screen's route. A claim about a transaction must state whether it was completed or only shown as form and dialog.

Where it wires in today: the bot reply path. The bot generator is deterministic, so the gate there is a proven guard, not a live filter catching model errors. The phase proves the gate works by feeding it tampered replies and confirming they are flagged, and by confirming zero false positives on a seeded set of valid replies. The gate then becomes mandatory for any future model call site: no model output ships in Centher without passing G2 first. That rule is written into the Phase 3 spec and the crew rules file.

Judge scores (specificity, helpfulness, tone) are signal, not proof, and are always reported separately from the four deterministic gates. A high judge score never overrides a failed gate.

G3: Model-call gateway (deferred, with a trigger)

Centher has no real model call sites today (verified by code search, see above). G3 is therefore not built in this plan. Building a gateway with no call sites would be infrastructure without a user.

Trigger condition: if a real model call site lands in Centher (an AI feature, a model-backed reply, a summary, or a transcription call inside this repo), a G3 gateway phase becomes mandatory before that feature ships. That phase will follow the shared contract: one server module for all model calls, traces in our own table (input, output, model, provider, tokens, latency, cost), SHA-256 content-keyed caching with exact-hash only, cache hits still emitting traces, and prompt versions stored with their schema version. Semantic caching stays deferred even then.

Note on voice: VoiSpace audio and video run through an external voice server. If transcription or any model call ever runs inside the Centher repo, the trigger above applies. If it runs only in the external voice server, it is outside this plan's scope, and the Centher chat must say so explicitly rather than assuming coverage.

G4: Spec discipline (adopted, Phase 4)

The rules, inherited from 00 and already close to Centher practice:

- Every feature starts as a short spec stating what it does, what it must not do, and how success will be checked. The spec lives in the repo or goal docs before building starts.
- If a build fails its acceptance check, discard the code, fix the spec, and rerun. Do not prompt on top of bad code. A failed acceptance means the spec was wrong.
- When ambiguous, the agent asks one specific question and brings a proposed answer (option A or B). Open-ended questions stall loops. Guesses create silent errors.
- The rules file for agents is kept short, because it is read on every prompt, and it avoids naming folders or structures the agent could then invent.
- Every spec names what is out of scope. For this system in version 1, out of scope everywhere: semantic caching, a visual graph canvas, quality judgments about code, and any stack migration.

Phase 4 writes these rules into the Centher crew docs and proves them on one real slice, so they are adopted practice rather than a document nobody reads.

G5: Tenant isolation (adopted as an audit, Phase 5)

Centher has tenant-like boundaries today: organizations and teams (orgs members and invites APIs), citizen versus non-citizen gating, and an admin surface (admin routes plus AdminGuard). This plan adopts G5 as a scoping audit, not as a rewrite.

- Every tenant-scoped row should carry a tenant id (organization id where the row belongs to an org). Every query should be scoped to it. The audit checks this claim against the real schema and the real API routes, and reports every row type and route where scoping is missing, assumed, or only enforced in the UI.

- Policy-first is the direction, adapted to this stack: the server derives the tenant from its own validated Better Auth session (never from request payload) and the audit states, per route, whether the server enforces that derivation or trusts a client-supplied value. Where the stack supports row-level policy, the audit names the tables where policy should be added. Where it does not, the audit names the server-side enforcement point that must exist instead. No policy mechanism is assumed from the video's Supabase pattern.
- Service keys and admin paths bypass policy by design and must never reach a client. The audit checks that admin routes and any service-key path are server-only, with their own audit trail.
- Acceptance test, copied from the shared system: two tenants, no cross-tenant rows, proven by switching tenants and reading. This test runs on demo or staging accounts only. If another tenant's row ever comes back, the bug is the policy (or the server enforcement point), not the screen.
- Cross-tenant rule for generated text: the G2 allowed sets are built per tenant, per request. A bot reply or any future model output for one tenant must never cite another tenant's data.
- If the audit finds a leak, the fix is a separate spec with its own Judge gate and Pilot. The audit phase itself does not silently rewrite access control.

Phases in order

Each phase follows the same gate order: PM writes the spec, the crew builds, QA runs Guardian tests, Judge gives APPROVE with numbers reproduced, Pilot passes on a live browser on the real surface, recorded, with a separate analysis pass before anything is called done. LEARNING.md is updated per phase. No phase merges on an agent completion report alone.

Crew assignments use the standing 12-agent crew. Role leads change per phase, but the gates do not: Engineering Manager gate on every phase, CEO agent gate only on pricing, positioning, public surfaces, and money flows (in Centher: registration fee, citizenship passport purchase, staking, marketplace trades, and any public-facing explainer output), and 3 client agents adapted to a dashboard product on every phase: a small-team admin (runs an org or team), an ops manager (runs marketplace or community operations), and a frontline user (posts, chats, and trades as a regular member). Client agents answer two questions per phase: would this solve your day, and would you pay. A client veto is recorded, not auto-blocking, and the Manager weighs it explicitly.

Phase 0: Verification and baselines (first, unblocks everything)

- Spec owner: Product Manager.
- Crew: PM writes the verification spec. Backend confirms repo facts (stack, routes, seed). QA records the baseline test result. Frontend confirms the demo sign-in path in a browser. Tech Lead owns the DayMirror core status check. DevOps/Release owns worktree and branch conventions (branch work in worktrees, primary checkout stays on main, no checkout of another branch over the working tree). Designer confirms nothing visual changes in this phase. Engineering Manager gates the baseline. CEO agent gate: not triggered (no pricing,

positioning, public surface, or money change). Client agents: confirm the demo accounts and flows they will test in later phases.

- Work: rebase the 75-screen inventory to current main (5c8a331c or later at run time) and record what changed since 8a33d5f1. Verify demo@centher.io sign-in in a live browser. Confirm or create the second demo account. Confirm the DayMirror core status (see Assumptions item 3) and record whether Centher ports an existing implementation or builds to the shared contract. Record the baseline Jest result and the baseline file counts. Update GOAL.md from the 5-agent loop description to the 12-agent crew.
- Acceptance checks: inventory rebase count stated (routes and modals, with additions and removals named). Demo sign-in proven in a browser, with the account shown. Second demo account confirmed or created, with its state recorded. DayMirror core status recorded in one paragraph. Baseline test numbers recorded (pass, fail, and known flakes, each failed file re-run standalone before being called a flake).
- Judge gate: Judge reproduces the inventory counts and the baseline test numbers from the repo, not from the report.
- Pilot: live browser sign-in as demo@centher.io on https://centher-app.vercel.app/, recorded, with a separate analysis pass. Read-only. Nothing saved, nothing deleted, no on-chain action.
- Risks: inventory drift between 8a33d5f1 and current main is larger than expected; demo seed data no longer matches the login box; the second demo account does not exist and must be created under demo rules.
- Definition of done: spec, rebase report, baseline numbers, demo account confirmation, and DayMirror core status are all in the goal docs. LEARNING.md gains a Phase 0 case (what was assumed before verification versus what the files actually showed).

Phase 1: G1 script (the core build)

- Spec owner: Product Manager, with Tech Lead owning the parser contract section.
- Crew: Backend builds the script. Tech Lead owns the graph JSON shape, edge kinds, coverage reporting, and the regression rule. QA owns the verification cases and the affected-tests query. Frontend advises on component roots and pages router naming (no UI is built in this phase). Designer: not triggered (no screen changes). DevOps/Release owns running the script in a repeatable way (one command, documented, runnable from a clean checkout). Engineering Manager gates. CEO agent gate: not triggered. Client agents: review the blast-radius output format for usefulness (can a non-engineer read what breaks).
- Work: build the script to the shared contract. Handle pages router suffixes, the @/* alias, and the component roots named above. Emit files, edges, and coverage as portable JSON. Implement blast-radius, dependency-chain, and affected-tests queries. Run it over the full repo. Report coverage next to every result.
- Acceptance checks: the script runs from one command on a clean checkout. Files found, parsed, and skipped are counted, with reasons for every skipped file. Imports resolved, failed, and skipped are counted, with reasons for every failed import. Three hand-verified blast-radius cases pass: a shared modal shell, the chat composer, and the feed post card, each verified by reading the actual imports, not by trusting the script. The affected-tests query returns the tests

importing the changed files, verified against a known change. Two consecutive runs on unchanged code produce identical edge lists.

- Judge gate: Judge re-runs the script, reproduces the counts, and re-checks at least one blast-radius case by hand from the code.
- Pilot: live browser check that the script's output matches the real surface for one case (for example, the screens the graph says import the shared post card are the screens where that card actually renders). Recorded, with a separate analysis pass. The Pilot proves the graph describes the real app, not just the code tree. No app code changes in the Pilot. Demo account only.
- Risks: pages router naming and the @/* alias complicate resolution; web3 and onchain code may import in ways the parser cannot resolve, which must be reported as failed with reasons, never guessed; the first full run over 827 files may be slow, which is acceptable if the run is repeatable and the time is recorded.
- Definition of done: script, JSON output, coverage report, and query results are in the repo or goal docs (the Centher chat decides the home, and records it). LEARNING.md gains a Phase 1 case (what the parser could not resolve, and why).

Phase 2: G1 applied to explainer ordering and onboarding (the Centher-specific payoff)

- Spec owner: Product Manager, with Designer co-owning the explainer ordering section.
- Crew: Backend provides the fan-in rankings from the Phase 1 graph. Designer owns the shared-components-first ordering map and reviews every screen and modal it touches through the UI/UX skills with local-ui-ux-director sign-off. Frontend advises on component boundaries. QA verifies the matrix against the rebased inventory. Tech Lead owns the matrix accuracy rule (every matrix cell must trace to a graph edge or be marked as assumed). DevOps/Release: not triggered beyond publishing the docs. Engineering Manager gates. CEO agent gate: triggered, because explainer output is a public surface. The CEO agent checks positioning and public claims only; new positioning decisions escalate to Irfan. Client agents: the small-team admin, ops manager, and frontline user each confirm the onboarding path makes sense for their role (which components they must understand first).
- Work: build the screen-to-component matrix for all 75 routes (rebased count from Phase 0). Rank shared components by fan-in. Produce the ordering map: shared components first, then screens grouped by the shared pieces they use. Write the onboarding guide: the components a new agent reads first, the screens each one serves, and the screens that can be skipped on first pass. Feed the ordering map to the explainer program as an input for any future re-record or new segment. Do not re-record the delivered explainer video in this phase. The delivered video stays as delivered.
- Acceptance checks: the matrix covers every route in the rebased inventory, with zero routes unmapped. Every matrix cell traces to a graph edge, or is marked as assumed with a reason. The top 10 shared components by fan-in are named, with their fan-in counts reproduced by the Judge from the Phase 1 output. The onboarding guide names a concrete first-read list (components, not screens) that covers the shared pieces serving the most screens. The

explainer ordering map states, per segment, which shared components that segment depends on.

- Judge gate: Judge reproduces the fan-in counts and spot-checks matrix cells against the code.
- Pilot: live browser walk of one onboarding path (read a top shared component's screens in the browser, in the order the guide names), recorded, with a separate analysis pass. Demo account only. No on-chain action. Destructive modals are shown then cancelled, never confirmed.
- Risks: the matrix may disagree with the explainer inventory where the inventory was code-only and the graph is also code-only; both are code evidence, so disagreements are resolved by reading the code, not by voting. The explainer video is already delivered, so this phase must not be presented as fixing the video; it improves future material and onboarding.
- Definition of done: matrix, ordering map, and onboarding guide are in the goal docs. LEARNING.md gains a Phase 2 case (which shared components dominate the app, and what that means for future changes).

Phase 3: G2 reference checker (the safety build)

- Spec owner: Product Manager, with Tech Lead owning the gate definitions.
- Crew: Backend builds the checker library (pure function, no model, no network). QA owns the adversarial test set (tampered replies that must be flagged) and the valid set (seeded replies that must pass). Frontend wires the checker at the bot reply path (as a guard, with flagging visible in logs, never silent rewriting). Designer: not triggered (no screen changes) unless the flagging surfaces in UI, in which case Designer reviews that surface. DevOps/Release owns running the checker in the test pipeline. Engineering Manager gates. CEO agent gate: not triggered, unless flagging text becomes user-visible public copy, in which case the CEO agent reviews that copy only. Client agents: confirm that flagged output handling is understandable (what happens when a reply is flagged, in plain language).
- Work: build the checker with the four gates and the Centher allowed sets defined above. Component names come from the G1 graph. Screen names come from the rebased inventory. Record ids come from a database read performed for that request. Entity names come from the app's own lists for that request. Numbers come from canonical app data for that request. Wire the checker at the bot reply path so every generated reply passes through it before delivery, with failures logged (failed gate, offending citation, allowed set checked against). Write the rule into the crew rules file: no future model output ships in Centher without passing G2 first.
- Acceptance checks: the adversarial set is fully flagged (every tampered component name, screen name, record id, entity name, and number is caught, with the correct gate named). The valid seeded set passes with zero false positives; if any false positive appears, the phase does not pass until the set definition or the checker is fixed and the reason is recorded. The bot reply path demonstrably routes through the checker (a test proves a flagged reply does not reach delivery, and the log entry names the gate and citation). The future-model rule is written in the crew rules file.

- Judge gate: Judge re-runs the adversarial and valid sets, reproduces the pass and flag counts, and inspects at least one log entry end to end.
- Pilot: live browser session as demo@centher.io where a bot reply is delivered through the guarded path, recorded, with a separate analysis pass. The Pilot proves the guard does not break normal chat. No tampered data is sent to a real user; tampering is proven in tests, not in the Pilot. Demo accounts only.
- Risks: record id validation needs a database read per request, which adds latency; the spec must state the latency budget and the caching rule for allowed sets (per request, per tenant, never shared across tenants). Entity lists may be incomplete for seeded data, which would cause false positives; the valid set must be built from real seeded data, not invented examples. Overclaiming risk: the gate proves references exist, not that the reply is correct, helpful, or safe; the phase report must state that limit plainly.
- Definition of done: checker library, test sets, wiring proof, log format, and the future-model rule are in the repo or goal docs. LEARNING.md gains a Phase 3 case (what the adversarial set caught that the valid set did not, and what the gate cannot prove).

Phase 4: G4 spec discipline adoption (the crew build)

- Spec owner: Product Manager.
- Crew: PM writes the crew rules file and the spec template. Tech Lead owns the discard-and-fix-spec rule wording and the acceptance check format. Designer owns the spec section for screen and modal review (every screen and modal through the UI/UX skills with local-ui-ux-director sign-off before clearing). QA owns the acceptance check template. Frontend and Backend each run one slice under the new discipline (see below). DevOps/Release owns the spec home (repo or goal docs, recorded). Engineering Manager gates. CEO agent gate: not triggered unless the slice touches a money or public surface, in which case that slice gets its own CEO check. Client agents: confirm the one-question-with-proposal escalation format works for their role (they receive one specific question with a proposed answer, not an open-ended question).
- Work: write the short crew rules file and the spec template into the Centher crew docs. Then prove them on one real slice: a small, real Centher fix or improvement chosen by the PM (for example, a known inventory gap such as a dead button, a placeholder screen, or a modal with no handler, chosen from the rebased inventory, not invented). The slice is built under the new discipline end to end: spec first, build, acceptance check, and a written record of whether the spec survived or was fixed after a failed acceptance. If no acceptance failure occurs, the record states that and still shows the acceptance evidence.
- Acceptance checks: the rules file exists, is short, and names no folders or structures the agent could invent. The spec template has what it does, what it must not do, how success will be checked, and what is out of scope. The slice has a spec written before code, an acceptance check with a recorded result, and a discard-or-fix record (either a fixed spec after a failure, or a statement that the spec survived with evidence). The escalation format is used at least once in the slice (a real ambiguity resolved by one specific question with a proposed answer) or the slice report states why no ambiguity arose.

- Judge gate: Judge reproduces the slice acceptance check and confirms the spec predates the code (from file history or the crew log).
- Pilot: live browser verification of the slice on the real surface (centher-app.vercel.app or the preview the Centher chat designates, recorded, with a separate analysis pass). Demo account only. Destructive modals cancelled. No on-chain action completed.
- Risks: the slice chosen may be too small to prove the discipline, or too large to finish in one phase; the PM must size it so the discipline is exercised, not just documented. The old 5-agent loop description in GOAL.md may confuse the crew; Phase 0 updates it, and Phase 4 must not proceed on the old description.
- Definition of done: rules file, spec template, slice spec, slice acceptance record, and escalation record are in the crew docs. LEARNING.md gains a Phase 4 case (where the discipline changed an outcome versus the old way of working).

Phase 5: G5 tenant scoping audit (the trust build)

- Spec owner: Product Manager, with Tech Lead owning the audit method.
- Crew: Backend audits the schema and API routes for tenant scoping. QA owns the two-tenant test. Frontend confirms which screens expose tenant data and where UI-only gating exists. Designer reviews any screen where the audit changes what is shown. DevOps/Release owns the demo and staging account setup for the test. Engineering Manager gates. CEO agent gate: triggered only if the audit touches money surfaces (registration fee, passport purchase, staking, marketplace trades) in its recommendations; the audit itself changes no money flow. Client agents: the small-team admin and ops manager confirm the org and team boundaries match how they actually run teams; the frontline user confirms they cannot see another team's data in the test.
- Work: audit every tenant-scoped row type and route. For each, record: the tenant id field (or its absence), whether the server derives the tenant from the validated Better Auth session or trusts a client-supplied value, whether enforcement is server-side or UI-only, and whether admin paths are server-only with an audit trail. Run the two-tenant test on demo or staging accounts: two tenants, switch tenants, read, and confirm no cross-tenant rows. Confirm G2 allowed sets are built per tenant, per request, in the Phase 3 wiring. If a leak is found, write a fix spec. Do not silently rewrite access control in the audit phase.
- Acceptance checks: the audit report covers every org, team, and admin route in the rebased inventory, with zero routes unexamined. The two-tenant test result is stated plainly (pass, or leak with the exact rows and routes). Every UI-only gating finding is named as UI-only, never described as enforcement. Any fix spec written from a finding has its own acceptance check and is tracked separately from the audit.
- Judge gate: Judge reproduces the two-tenant test (or re-runs the audit queries) and confirms the report matches the code.
- Pilot: live browser session switching between two demo or staging tenants, recorded, with a separate analysis pass, showing that each tenant sees only its own data. Demo or staging accounts only. No production data. No on-chain action.

- Risks: the second tenant account may not exist (see Assumptions item 1); Phase 0 must resolve this before Phase 5 starts. Admin routes may rely on middleware lists that drift from the code; the audit must check both the middleware list and the AdminGuard call. A leak finding may require a fix larger than one phase; that fix becomes its own plan, and this phase still completes as an audit with a fix spec.
- Definition of done: audit report, two-tenant test result, and any fix specs are in the goal docs. LEARNING.md gains a Phase 5 case (every boundary that was UI-only, and what server enforcement replaced or must replace).

Phase 6: Folder folding navigation view (later phase, only if approved)

- Spec owner: Product Manager. This phase does not start until Phases 1 and 2 are done and the Centher chat explicitly agrees the view is worth a UI surface.
- Crew: Designer owns the folding interaction and reviews every screen and modal through the UI/UX skills with local-ui-ux-director sign-off. Frontend builds the view from Phase 1 graph output only (no new parser logic). Backend provides the graph JSON endpoint or file the view reads (read-only, demo data only in Pilot). QA owns the navigation tests (fold, expand, select, clear). Tech Lead owns the rule that the view never invents an edge (every shown edge traces to the graph). DevOps/Release owns the preview deployment for the Pilot. Engineering Manager gates. CEO agent gate: triggered, because this is a new public-facing surface if it ships to users; if it ships as an internal crew tool only, the CEO agent confirms that scoping and the gate is recorded as such. Client agents: confirm the feature-folder navigation matches how they think about the app (by feature, not by file).
- Work: build a read-only navigation view over the Phase 1 graph: folders grouped, small folders merged upward until the map is manageable, clicking a folded folder lists its files, selecting a file shows its imports and imported-by lists. No AI explanation in version 1 of this view. No editing from the view.
- Acceptance checks: the view renders from Phase 1 output with no hand-written edges. Every edge shown traces to the graph. Folding reduces the visible node count to the number the spec names, measured, not estimated. Selecting a file shows its blast radius and dependency chain from the graph, matching the script queries for the same file.
- Judge gate: Judge reproduces the node counts and spot-checks edges against the graph output.
- Pilot: live browser walk of the view on the real surface, recorded, with a separate analysis pass. Demo account only. No on-chain action. Destructive actions do not exist in this view by design.
- Risks: the view may be slower or less useful than the script and matrix; if the Pilot shows that, the phase report says so and the view is not shipped. Building a canvas before the script is proven is the exact mistake this ordering avoids.
- Definition of done: view (or a written decision not to ship it, with Pilot evidence) plus LEARNING.md case.

G3 trigger, stated once for the whole plan

If a real model call site lands in Centher, the Centher chat writes a G3 phase before that feature ships, following the shared contract (one server module, own traces table, SHA-256 content-keyed caching with exact-hash only, cache hits still traced, prompt versions stored with schema versions). Until then, G3 stays deferred, and no plan, report, or demo claims Centher has a model gateway.

Relationship to the explainer program

This plan supports the explainer program. It does not replace it.

- The explainer video is delivered (775.03 seconds, 19 segments, 140 items covered with honest qualifiers). Nothing in this plan re-records it, re-edits it, or changes its verdicts.
- Phase 2 feeds the explainer program two inputs for future material: a shared-components-first ordering map and a screen-to-component matrix. If a future re-record or new segment is planned, it uses that ordering. If no future recording is planned, the matrix still serves onboarding and change planning.
- The explainer checklist's honest qualifiers (unreachable controls, disabled listing buttons, unopened staking detail pages, note-only modals) remain open product gaps. This plan does not close them by itself. Where a future phase fixes one, that fix gets its own spec, Judge gate, and Pilot, and the checklist is updated from the new evidence, not from this plan's claims.
- The VoiSpace rule stands: hide the button if audio or video is not working, keep it if it works. This plan does not verify VoiSpace live, and no phase assumes it works.

Global risks

- Inventory drift: the explainer inventory base (8a33d5f1) is behind current main (5c8a331c). Phase 0 exists to close that gap before any later phase relies on inventory counts.
- Demo account drift: seed data, login box values, and the second demo account may not match. Phase 0 verifies by signing in.
- Parser risk: pages router naming, the @/* alias, and web3 imports may produce failed resolutions. Those are reported with reasons, never guessed, and the coverage report makes them visible.
- Overclaiming risk: the graph proves structure, not correctness. G2 proves references exist, not that text is true. The bot guard proves routing, not intelligence. Every phase report must state these limits plainly.
- On-chain risk: no on-chain action is verifiable from code alone, and none is completed in any Pilot. Any phase report that claims an on-chain result without a live transaction record is wrong and must be corrected in the same report.
- Parallel rollout risk: DayMirror, Clinic, Viyacor, Elite JS, and Viva plans proceed in their own chats. If any of them changes the shared contract in 00, the Centher chat must reconcile before Phase 1 code is written. Phase 0 records the contract version it builds against.

Definition of done (plan level)

The plan is done when every phase above has: a spec written before code, a Judge APPROVE with numbers reproduced, a live-browser Pilot on the real surface, recorded, with a separate analysis pass that came back clean, LEARNING.md updated with that phase's cases, a learning note and PDF, and test data cleaned up with ids recorded. A phase that has only a test suite result, a database write, a merge, or an agent completion report is not done. The standing rule applies: those are claims, not proof.

PDF conversion of this plan is the parent step. This document is the Markdown source. Implementation starts in the Centher chat from this plan, not in this document.

What happens next

1. The parent delivers this plan (Markdown plus PDF) to the user.
2. The Centher chat picks up Phase 0 from this plan. No implementation starts here.
3. Phase 0 resolves the assumptions (second demo account, DayMirror core status, inventory rebase) before Phase 1 code is written.
4. Each phase runs in the Centher chat under the 12-agent crew, with the gates in this plan.